

1장

1) 머신러닝 vs 딥러닝

- 머신러닝(ML): 데이터를 바탕으로 규칙을 "학습"해 예측/분류를 수행하는 알고리즘 총칭. 지도·비지도·강화학습으로 나뉘며, 문제 정의 → 데이터 수집/정제 → 특징 설계(특징 공학) → 모델 학습/평가 → 배포의 절차로 진행되는 파이프라인을 갖습니다.
- 딥러닝(DL): 인공신경망을 여러 층으로 깊게 쌓아 특징 추출+예측을 통합적으로 학습합니다(엔드-투-엔드). 역전파와 경사하강으로 가중치를 갱신하며, 대규모 데이터·연산에서 강점을 보입니다.

한 줄 비교: ML=사람이 특징을 많이 설계, DL=모델이 특징까지 같이 학습. (1장은 이 관점을 토대로 이후 장에서 파이토치로 구현하는 흐름을 예고합니다.)

2) 딥러닝 학습 흐름(요약)

- 1) 데이터 준비(훈련/검증/시험 분할) → 2) 모델 정의(층 쌓기) → 3) 손실 함수 선택 → 4) 옵티마이저로 경사하강(역전파) → 5) 평가지표 확인/튜닝. 책 1.3절은 특히 "순전파 → 손실 계산 → 역전파 → 가중치 갱신"의 루프를 핵심으로 설명합니다.

3) 활성화 함수(Activation Functions)

왜 필요한가? 선형 변환만으로는 비선형 패턴을 못 잡습니다. 활성화는 비선형성을 주어 복잡한 함수를 근사하도록 만듭니다.

자주 쓰는 함수와 특성

- ReLU: $\max(0, x)$ — 계산이 빠르고 기울기 소실이 적어 CNN 등 기본값처럼 널리 사용
- Sigmoid: $\sigma(x)=1/(1+e^{-x})$ — (0,1) 범위. 이진 확률 해석에 유용하나 포화 구간에서 기울기 소실 가능.
- Tanh: (-1,1) 범위의 S자 곡선. 평균 0에 가까워 수렴이 나아질 수 있음.
- Softmax: 각 클래스의 확률 분포로 변환(각 축 합=1). 보통 **출력 층의 로짓(logits)**에 적용된 것으로 해석(크로스엔트로피와 결합).

4) 손실 함수(Loss Functions)

정의: 모델 예측과 정답의 차이를 수치화해 학습 방향을 제시합니다. 문제 유형별로 다른 손실을 씁니다.

- 회귀 → `nn.MSELoss` : 평균제곱오차. 예측과 타깃 차이의 제곱 평균을 최소화.
- 다중 분류 → `nn.CrossEntropyLoss` :
 - 입력: 소프트맥스를 거치지 않은 로짓(크기 `[N, C]` 혹은 `[N, C, d1, ...]`).
 - 타깃: 각 샘플의 정수 클래스 인덱스(크기 `[N]` 또는 `[N, d1, ...]`).
 - 내부적으로 `log_softmax + NLLLoss`를 합쳐 확률을 직접 넣지 않습니다.
- 이진/멀티라벨 → `nn.BCEWithLogitsLoss` : 시그모이드+BCELoss를 하나로 합친 형태(수치적으로 안정적). 로짓을 그대로 넣고 타깃은 `{0,1}` 실수 텐서.

5) 데이터의 형태(파이토치에서 자주 만나는 텐서 규약)

- 배치 차원: `DataLoader`가 샘플들을 묶어 **맨 앞 축에 `N`(배치 크기)**를 붙입니다. (예: 특성 10짜리 표형 데이터 → `[N, 10]`)
- 이미지: 기본 `NCHW`(배치, 채널, 높이, 너비) 형식이 표준입니다. 연산 최적화를 위해 채널-라스트(`NHWC`) 메모리 포맷도 지원되지만, 연산자 기본 기대는 채널-퍼스트입니다.
- 텍스트/시계열: 흔히 `[N, L]`(길이 `L` 시퀀스) 또는 임베딩을 쓰면 `[N, L, D]`.
- 분류 타깃의 모양
 - `CrossEntropy`: 정수 인덱스, dtype Long, 크기 `[N]`(또는 픽셀 단위 분류면 `[N, H, W]`).
 - `BCEWithLogits`: 실수 `{0,1}`, 크기 입력과 동일.

TensorBoard/시각화 등에서 이미지 기본 텐서는 `(N,3,H,W)`로 가정됩니다.

2장

1. 파이토치 개요

1.1 파이토치란?

- 페이스북 AI 리서치팀에서 개발한 딥러닝 프레임워크.
- Python 친화적이며, 연구/산업 현장에서 많이 사용됨.
- 핵심 장점:
 - GPU 가속: CUDA 지원으로 연산 속도 향상.
 - 자동 미분(autograd): 역전파를 자동으로 처리.
 - 동적 계산 그래프(Define-by-Run): 실행 시점에 그래프를 생성해 직관적이고 디버깅이 쉬움.

2. 파이토치의 구조

2.1 아키텍처 레벨

- Tensor: 다차원 배열, Numpy 배열과 유사하지만 GPU 가속 가능.
- Autograd: 텐서 연산을 추적해 미분(기울기) 자동 계산.
- nn.Module: 신경망 구조를 정의하는 클래스.
- optim: SGD, Adam 등 최적화 알고리즘 제공.
- torchvision, torchaudio, torchtext: 이미지/음성/텍스트 데이터 지원 라이브러리.

2.2 동적 계산 그래프

- 실행할 때마다 그래프를 새로 정의 → 유연한 모델 설계 가능.
- TensorFlow 1.x의 정적 그래프 방식과 대비됨.

3. 파이토치 기초 문법

3.1 텐서(Tensor)

- 정의: Numpy의 ndarray와 유사한 다차원 배열.

- 특징: GPU(CUDA) 사용 가능 → `.to('cuda')` 또는 `.cuda()`.
- 텐서는 신경망의 입력, 가중치, 출력 등 거의 모든 데이터 표현에 사용.

3.2 자동 미분(Autograd)

- `requires_grad=True` 속성을 가지는 텐서는 연산 기록 추적.
- `.backward()` 호출 시 자동으로 기울기 계산.
- 계산된 기울기는 `.grad` 속성에 저장.

4. 신경망 구성 요소

4.1 모델 정의

- 모든 모델은 `nn.Module`을 상속받아 정의.
- `__init__`: 층과 연산 정의.
- `forward`: 순전파 연산 정의.

4.2 손실 함수

- 회귀: 평균제곱오차(MSELoss).
- 분류: 크로스엔트로피(CrossEntropyLoss).
- 손실 함수는 예측값과 정답 간 오차를 수치화하여 학습 목표를 제시.

4.3 최적화 함수

- 파라미터 갱신 알고리즘.
- SGD, Adam, RMSprop 등 다양한 알고리즘 제공.
- 파라미터는 `model.parameters()`로 전달.

5. 데이터 다루기

5.1 Dataset & DataLoader

- Dataset: 데이터셋 클래스, `__getitem__`, `__len__` 구현 필요.
- DataLoader: 배치 단위로 데이터 제공, 셔플 가능.

5.2 배치와 에포크

- 배치(batch): 학습에 사용하는 데이터 묶음.
- 에포크(epoch): 전체 데이터셋을 한 번 모두 학습하는 주기.

6. 학습 과정(훈련 루프)

1. 순전파(forward): 입력 → 모델 → 출력 계산.
2. 손실(loss): 예측과 정답 차이를 손실 함수로 계산.
3. 역전파(backward): 손실을 기반으로 기울기 계산.
4. 최적화(step): 옵티마이저로 파라미터 갱신.
5. 이 과정을 반복하며 성능 향상.

7. 실습 환경 설정

7.1 Anaconda 가상환경

- 파이썬 패키지 충돌 방지를 위해 권장.
- 프로젝트별로 독립 환경을 만들어 관리.

7.2 파이토치 설치

- 공식 웹사이트에서 CUDA 버전에 맞는 설치 명령어 제공.
- 예시:
- `conda install pytorch torchvision torchaudio cudatoolkit=11.3 -c pytorch`

7.3 개발 도구

- Jupyter Notebook: 실습과 시각화에 적합.
- TensorBoard: 학습 곡선, 손실 추이, 그래프 시각화 가능.